PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

# *Incognito Campus*

Submitted by:



| | | |
|---|---|---|
| PES1UG24AM097 | Diya B R | 5th Semester |
| PES1UG24AM120 | Janavi Agrawal | 5th Semester |

# Abstract

Incognito Campus is a campus question-and-answer application that helps students ask academic doubts, find existing explanations, and receive answers from peers and verified teachers. Students appear under anonymous handles in public Q&A, teachers appear under their verified display names, and role checks protect teacher and administrator functions. PostgreSQL 16 stores the relational records and the pgvector extension stores 384-dimensional post embeddings (all-MiniLM-L6-v2) for semantic search and duplicate detection. A Node.js and Express API connects the database to a lightweight HTML/CSS/JavaScript browser interface, and a Python Flask service generates the embeddings; if that service is unavailable, search falls back to keyword matching and posting continues to work. Moderation rules, including report review and duplicate merging, are enforced inside the database through functions and triggers.

This report documents the work done by the two team members. Diya B R built the core platform: the relational schema, authentication and role-based access, the question and answer lifecycle (doubts, answers, comments, votes, tags and accepted answers), SQL queries and the main web screens. Janavi Agrawal built the intelligence and moderation layer: pgvector embeddings, semantic search, duplicate detection and merging, report moderation, analytics dashboards, index tuning and the test scripts.

**Table of Contents**


| No. | Section | Page |
|---|---|---|
| 1 | Introduction and problem statement | 0 |
| 2 | User requirements and technology stack | 0 |
| 3 | E-R model and entities | 0 |
| 4 | Relational model and schema diagram | 0 |
| 5 | SQL DDL statements | 0 |
| 6 | SQL DML statements | 0 |
| 7 | Vector database commands | 0 |
| 8 | Results and demonstration | 0 |
| 9 | Project tree structure | 0 |
| 10 | Application flow and code | 0 |
| 11 | Evaluation rubric and status | 0 |

# Introduction

## Problem Statement

Students often repeat the same course questions across informal chats and separate message threads. Helpful answers are difficult to find later, students may hesitate to ask publicly, and moderators need clear controls for handling inappropriate content. Incognito Campus provides one searchable, persistent Q&A space organized by subject and topic.

## Project Description

The application supports anonymous student participation, answers from verified teachers, voting, comments under answers, tags, accepted answers, semantic search, live duplicate suggestions while a doubt is being written, duplicate merging, report review and moderation dashboards. A student can also mention a teacher in a doubt and add a new subject or topic when none fits. PostgreSQL remains the source of truth for users, doubts, answers, votes, reports, and access rules.

Novelty comes from combining privacy-aware campus Q&A with semantic duplicate discovery and a moderation trail. The vector index helps find questions by meaning, while ordinary SQL handles transactions, constraints, joins, and analytics.

# User Requirements Specification

## Functional Requirements

Users can sign in by campus email or university ID and browse and search doubts (semantic search, with a keyword fallback). Students can create a doubt under a subject and topic (choosing an existing one or adding a new subject or topic), tag it, and optionally mention a teacher. Students and verified teachers can post answers; any signed-in user can comment under answers, vote, and report a doubt or answer. The asker or a verified teacher can accept or unaccept an answer, and, when authorized, admins and verified teachers can review reports, merge duplicates and view the moderation dashboard.

## Roles and Privacy

The application supports student, teacher, and admin roles. Students appear under an anonymous handle in public Q&A. Teachers use a profile display name, and only verified teachers can use teacher-only actions. Only students can post doubts; students and verified teachers can post answers. Admins and verified teachers can moderate. The API middleware and database functions both check moderator eligibility.

## Technology Stack and Vector Search

PostgreSQL 16 is the primary RDBMS. It provides foreign keys, check constraints, transactions, joins, aggregations, functions, triggers and indexes. pgvector adds vector storage and cosine-distance search inside PostgreSQL; the project therefore demonstrates relational and vector database features without claiming a separate MongoDB store. Node.js with Express 5 exposes parameterized API routes using pg, with jsonwebtoken and bcryptjs for authentication; vanilla HTML/CSS/JavaScript renders the browser interface. A Python Flask service uses sentence-transformers (all-MiniLM-L6-v2, 384 dimensions) to generate embeddings and to run semantic search and duplicate checks (cosine-similarity threshold 0.85 for duplicates). The database runs in Docker from the pgvector/pgvector:pg16 image.

## Individual Contribution

Diya: Core platform: schema, authentication, role-based access, posts, answers, comments, votes, web UI

Janavi : Intelligence and moderation: pgvector, embeddings, semantic search, duplicates and merge, reporting and moderation, dashboards, indexes, tests

# E-R Model

## Entities and Dependent Relations

Core entities are users, student, teacher, subject, topic, post, post_embeddings, tag, answer, comment, votes, and report. The student and teacher profile rows depend on a user. post_embeddings depends on its post. teaches and post_tag are associative relations. topic is identified within a subject by the composite key (subject_id, topic_name). A post may also reference a mentioned teacher (mentioned_teacher_id) and, once merged as a duplicate, the original post (merged_into).

## Entity Relationship Diagram

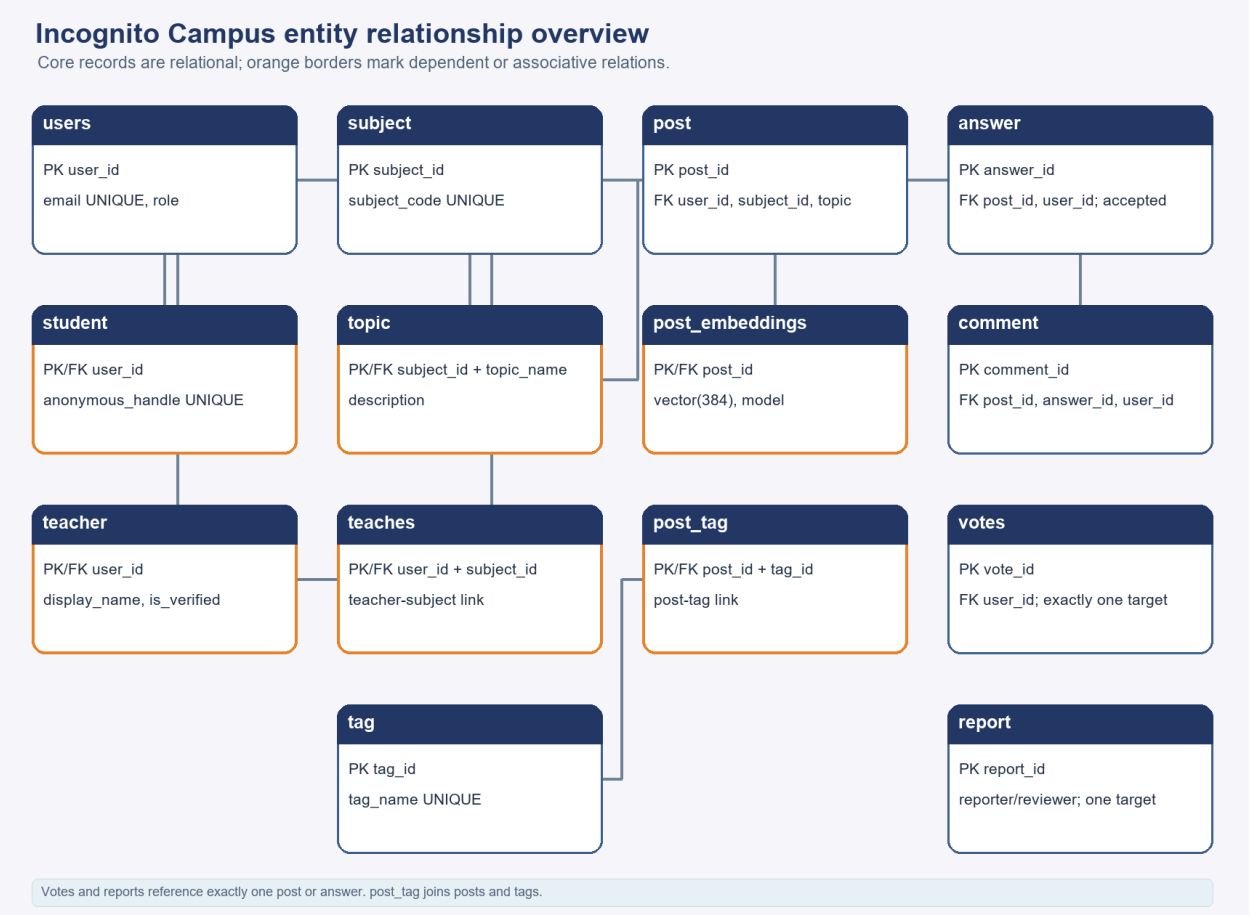



Figure 1. Entity relationship overview based on schema.sql. Orange borders mark dependent and associative relations.

# Relational Model

The implementation contains fourteen relations. Primary keys identify each row; foreign keys connect identities, subjects, doubts, answers, comments, votes, tags, embeddings, and reports. Migration 001 adds post_embeddings.post_id as the primary key and a self-reference (merged_into) for merged doubts; migration 002 adds the moderation functions, trigger and thread view; and post.mentioned_teacher_id, which links a doubt to a mentioned teacher, is defined in schema.sql and also applied by migration 004 for existing databases. Migrations 001 to 003 are not yet folded into schema.sql, so they must be loaded after schema.sql and seed.sql.

### Schema Diagram and Relation Inventory

| Relation | Primary key | Foreign keys and purpose |
|---|---|---|
| users | user_id | Base identity, email, password hash, role |
| student | user_id | FK users.user_id; anonymous handle profile |
| teacher | user_id | FK users.user_id; display name and verification |
| subject | subject_id | Subject code/name and department |
| topic | subject_id + topic_name | FK subject.subject_id; subject-scoped topic |
| teaches | user_id + subject_id | FK teacher.user_id and subject.subject_id |
| post | post_id | FK user_id, subject_id, composite topic key, mentioned_teacher_id (teacher) and merged_into (post) |
| post_embeddings | post_id | FK post.post_id; vector(384), model name and timestamp (primary key added by migration 001) |
| tag | tag_id | Unique tag_name |
| post_tag | post_id + tag_id | FK post.post_id and tag.tag_id |
| answer | answer_id | FK post.post_id and users.user_id |
| comment | comment_id | FK post.post_id, answer.answer_id, users.user_id |
| votes | vote_id | FK users.user_id, post.post_id, answer.answer_id; exactly one post or answer target; one vote per user per target |
| report | report_id | FK reporter/reviewer users, post.post_id, answer.answer_id; exactly one target; reviewer set only once decided |

Integrity rules include unique emails, anonymous handles, teacher display names, subject codes and tag names; roles limited to student, teacher, or admin; doubt states limited to open, resolved, or closed; votes restricted to -1 or 1; and checks requiring each vote and report to target exactly one post or answer. Unique constraints and indexes prevent a user from repeating a vote or a report against the same target, a doubt cannot be merged into itself, and a report has a reviewer if and only if it is no longer pending. The report_rules trigger blocks self-reports and requires the reviewer to be an admin or verified teacher who is neither the reporter nor the author of the reported content; review_report records the reviewer and prevents reopening a decided report.

SQL DDL Statements

Core tables are defined in schema.sql. Migrations 001 and 002 add search, merge and moderation constraints, functions and triggers; 003 adds measured indexes and 004 adds the teacher-mention reference.

```
CREATE TABLE report (
  report_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reporter_user_id VARCHAR(20) NOT NULL REFERENCES users(user_id),
  reviewer_user_id VARCHAR(20) REFERENCES users(user_id),
  post_id UUID REFERENCES post(post_id) ON DELETE CASCADE,
  answer_id UUID REFERENCES answer(answer_id) ON DELETE CASCADE,
  reason TEXT NOT NULL,
  status VARCHAR(10) NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','actioned','dismissed')),
  created_at TIMESTAMP NOT NULL DEFAULT now(),
  CHECK ((post_id IS NULL) <> (answer_id IS NULL))
);

-- Migration 001: reviewer is set only once decided; one report per user per target
ALTER TABLE report ADD CONSTRAINT chk_reviewer_matches_status
  CHECK ((status = 'pending') = (reviewer_user_id IS NULL));
CREATE UNIQUE INDEX uq_report_user_post ON report (reporter_user_id, post_id)
  WHERE post_id IS NOT NULL;

-- Migration 001: merge workflow (a duplicate points at its original)
ALTER TABLE post ADD COLUMN merged_into UUID REFERENCES post(post_id) ON DELETE SET NULL;
ALTER TABLE post ADD CONSTRAINT chk_no_self_merge
  CHECK (merged_into IS NULL OR merged_into <> post_id);

CREATE INDEX idx_post_status ON post(status, created_at DESC);
```

SQL DML Statements

```
-- Joins with CTEs (condensed from queries.sql): answers and vote totals per doubt.
-- Each total is aggregated before joining so rows are not double-counted.
WITH answer_counts AS (
  SELECT post_id, count(*) AS answer_count FROM answer GROUP BY post_id
), post_vote_totals AS (
  SELECT post_id, sum(vote_value) AS vote_score
    FROM votes WHERE post_id IS NOT NULL GROUP BY post_id
)
SELECT p.post_id, p.title,
       COALESCE(ac.answer_count, 0) AS answer_count,
       COALESCE(pvt.vote_score, 0) AS vote_score
  FROM post p
  LEFT JOIN answer_counts ac ON ac.post_id = p.post_id
  LEFT JOIN post_vote_totals pvt ON pvt.post_id = p.post_id
 ORDER BY p.created_at DESC;

-- Correlated subquery (queries.sql): vote score of each answer on one doubt
SELECT a.answer_id, a.is_accepted,
       COALESCE((SELECT sum(v.vote_value) FROM votes v
                 WHERE v.answer_id = a.answer_id), 0) AS vote_score
  FROM answer a
 WHERE a.post_id = '40000000-0000-0000-0000-000000000001'
```

```
 ORDER BY a.is_accepted DESC, vote_score DESC;

-- NOT EXISTS (queries.sql): open, unanswered doubts in a teacher's subjects
SELECT p.post_id, p.title
  FROM teaches teach JOIN post p ON p.subject_id = teach.subject_id
 WHERE teach.user_id = 'FAC001' AND p.status = 'open'
   AND NOT EXISTS (SELECT 1 FROM answer a WHERE a.post_id = p.post_id);

-- Moderation outcomes
SELECT status, COUNT(*) AS reports
FROM report GROUP BY status ORDER BY status;
```

Vector Database Commands

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE post_embeddings (
  post_id UUID NOT NULL REFERENCES post(post_id) ON DELETE CASCADE,
  embedding vector(384),
  embedding_model VARCHAR(100),
  embedding_updated_at TIMESTAMP,
  CHECK ((embedding IS NULL) = (embedding_model IS NULL)
     AND (embedding IS NULL) = (embedding_updated_at IS NULL))
);

-- Migration 001: one embedding row per post
ALTER TABLE post_embeddings ADD PRIMARY KEY (post_id);

CREATE INDEX idx_post_embedding_hnsw
  ON post_embeddings USING hnsw (embedding vector_cosine_ops);

-- $1 is the query embedding generated by the Python service.
-- Similarity = 1 - cosine distance; 0.4 for search, 0.85 for duplicate checks.
SELECT p.post_id, p.title, 1 - (e.embedding <=> $1) AS similarity
  FROM post_embeddings e JOIN post p ON p.post_id = e.post_id
 WHERE e.embedding IS NOT NULL AND p.merged_into IS NULL
   AND 1 - (e.embedding <=> $1) >= 0.4
 ORDER BY e.embedding <=> $1
 LIMIT 5;
```

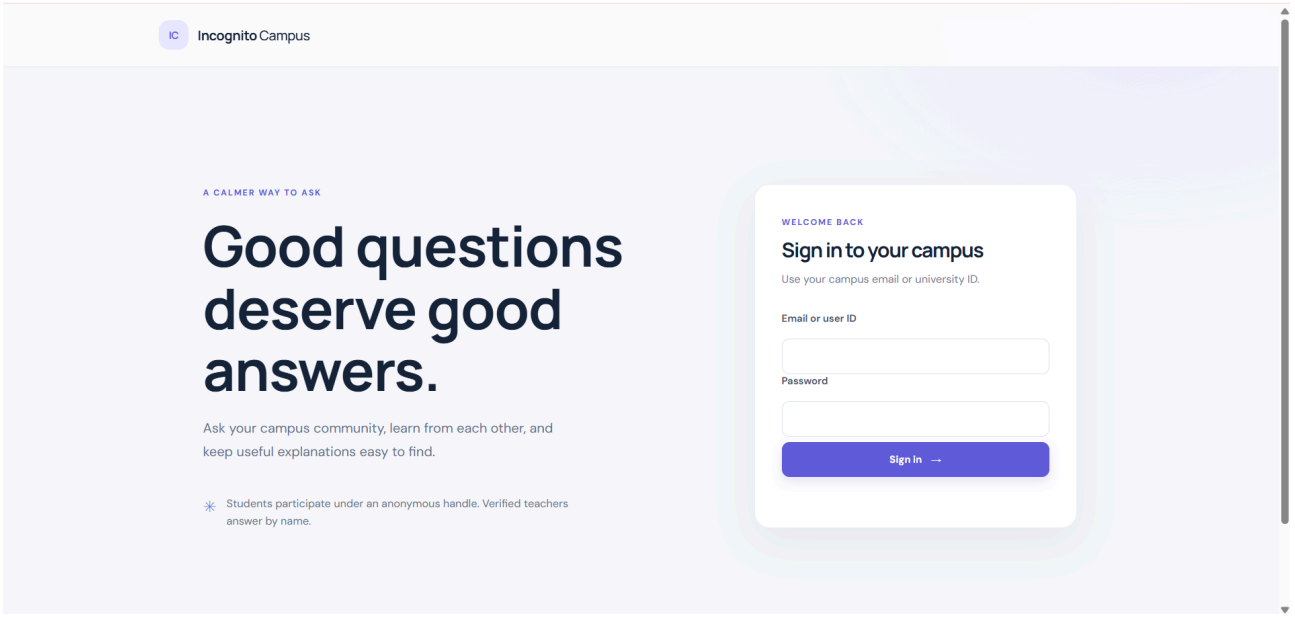
## Results and Demonstration

The repository implements the workflows below. The project includes SQL and API test scripts, but this report does not claim a test run or database row counts. Capture live result grids and application screenshots from the configured demo database during the presentation.

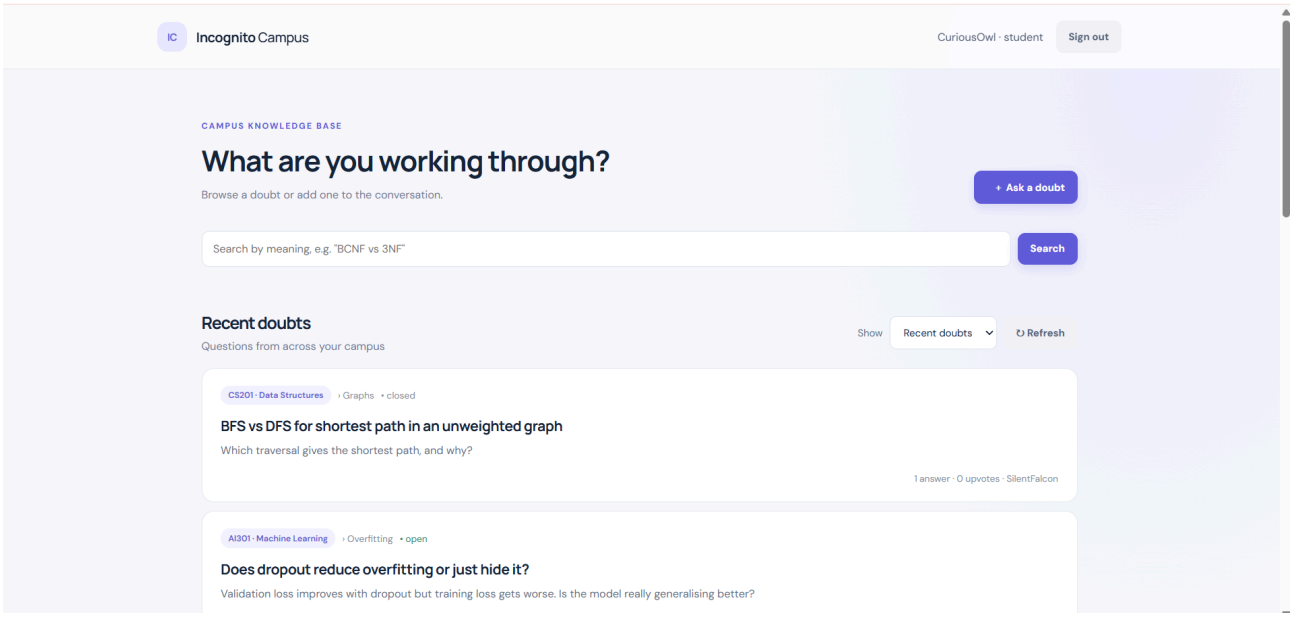
| Feature | Implementation evidence | Demo action |
|---|---|---|
| Anonymous Q&A | student.anonymous_handle; role-based public author name | Sign in as student and view a doubt |
| Answer lifecycle | answer.is_accepted; accept/unaccept transactions | Add an answer, accept it, then undo acceptance |
| Moderation accountability | report.reviewer_user_id, report_rules trigger and review_report() | Dismiss a test report and show reviewer history |
| Semantic retrieval | vector(384) column and HNSW cosine index | Search by meaning and inspect a suggestion |
| Analytics | dashboard queries and moderation API | Show outcomes and peak doubt dates |
| Duplicate handling | merge_post(); post.merged_into; v_thread_answers view | Mark a doubt as duplicate and open the original thread |
| Index tuning | idx_report_pending_created and idx_post_recent_unmerged, measured with EXPLAIN ANALYZE on generated data (tests/bulk_data.sql) | Run queries/explain_demo.sql before and after migration 003 |

## Application ScreenShots:

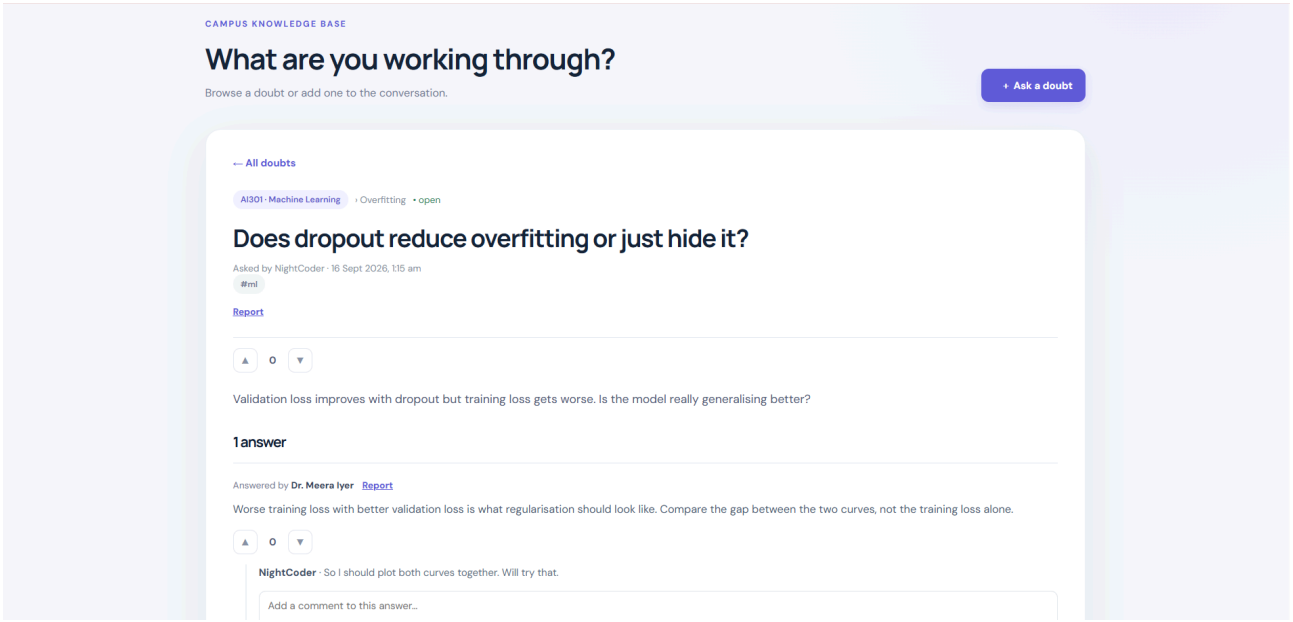
### Login page:



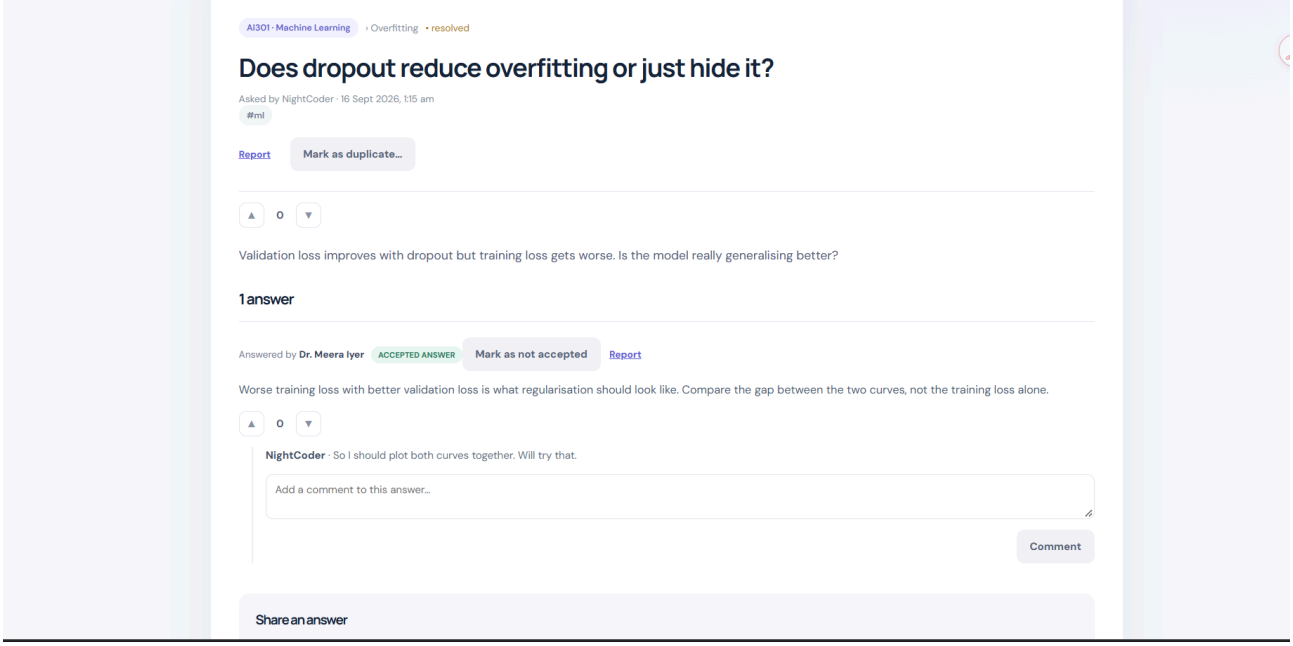
## Student Dashboard:



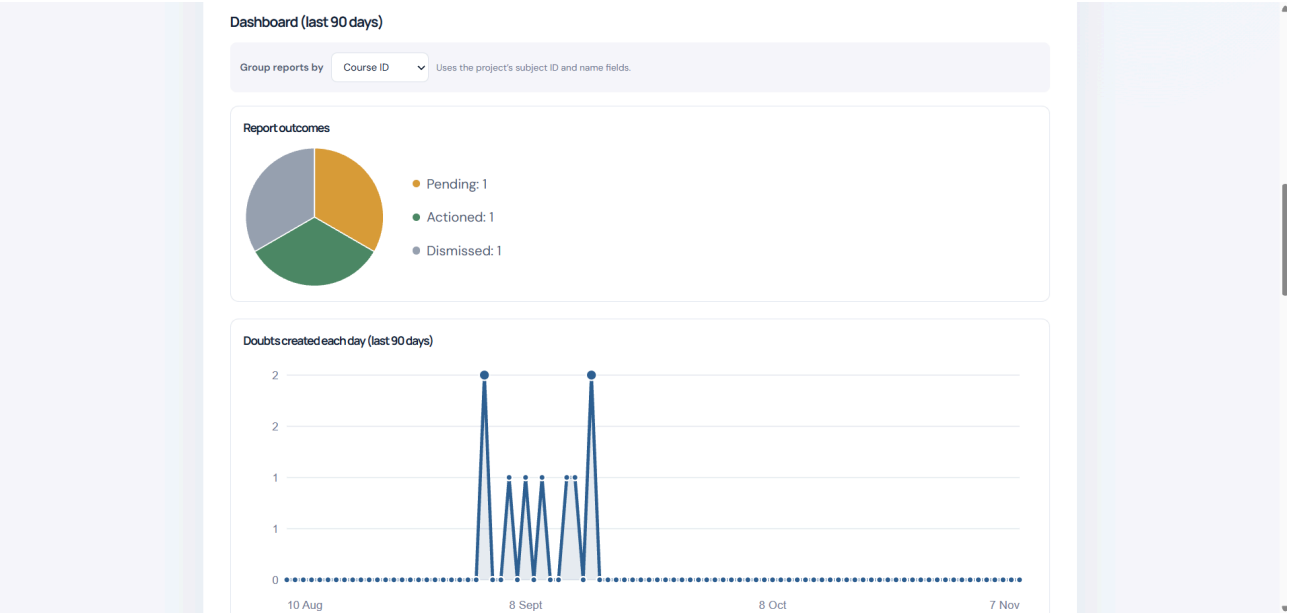

## Question page:



## Page showing Accepted answer:




## Report Page:



## Project Tree Structure

This template requests a Next.js tree, but the checked-in application uses Express with a static vanilla JavaScript frontend. The following reflects the actual project:

```
Incognito-Campus/
|-- schema.sql, seed.sql, queries.sql, README.md
|-- migrations/ 001_search_moderation.sql, 002_merge_and_moderation.sql,
|               003_dashboards_indexes.sql, 004_teacher_mention.sql
|-- queries/ dashboards.sql, explain_demo.sql
|-- src/ server.js, db.js, middleware/requireAuth.js
|   |-- routes/ auth, posts, answers, comments, subjects, teachers, search, moderation
|   `-- services/ searchClient.js, subjectService.js
|-- public/ index.html, app.js, extras.js, styles.css, extras.css
|-- embed_posts.py, search_service.py, requirements.txt
|-- package.json, .env.example
`-- tests/ test_edge_cases.sql, test_merge_moderation.sql, bulk_data.sql,
          smoke_integration.js, test_custom_subject.js, test_teacher_topic.js
```

# Application Flow and Code

### Authentication and Role Checks

POST /api/auth/login accepts a campus email or user ID, reads the account and profile, compares the password with bcrypt, and signs a short-lived JWT (HS256, two hours by default) containing the user ID, the role and, for teachers, the verification flag. API middleware (requireAuth, requireRole, requireVerifiedTeacher and requireAnswerer) checks the token and role before protected actions: only students can create doubts, and only students and verified teachers can answer. Student authors appear under an anonymous handle; teachers appear under their display name.

### Accepted Answer Transaction

The accept route begins a transaction, locks the doubt row, confirms that the caller is the asker or a verified teacher, updates the answer, updates the doubt status, and commits. If validation fails, it rolls back, and a closed (merged) doubt cannot accept an answer. The unaccept route also runs in a transaction and recalculates whether the doubt remains resolved. A doubt may have more than one accepted answer.

```
BEGIN;
SELECT user_id, status FROM post WHERE post_id = $1 FOR UPDATE;
UPDATE answer SET is_accepted = TRUE
 WHERE answer_id = $2 AND post_id = $1;
UPDATE post SET status = 'resolved' WHERE post_id = $1;
COMMIT;
```

### Application Screens and Testing

The browser interface includes sign-in, the searchable doubt feed with semantic search, a doubt form with live duplicate suggestions, teacher mentions and custom subject or topic entry, question detail with answers and comments, voting, accepted-answer controls, reporting, and the moderation dashboard with its review queue and a Mark as duplicate action. Use the local demo to capture screenshots of the feed and question detail after signing in with the demo accounts. Automated checks are in tests/ (test_edge_cases.sql, test_merge_moderation.sql, smoke_integration.js, test_custom_subject.js and test_teacher_topic.js).

# Evaluation Rubric and Completion Status

| # | Criteria | Distribution | Status |
|---|---|---|---|
| 1 | Database Design | ER diagram , relational schema with keys, relationships, and constraints | Implemented: 14-table schema, keys, relationships, constraints, indexes, and ER overview. |
| 2 | Database Creation | SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification | Implemented: PostgreSQL + pgvector schema and migrations; vector storage is a PostgreSQL extension. |
| 3 | SQL Operations and Queries | DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries | Implemented in SQL sources: joins, aggregations, CTE/dashboard queries, correlated and NOT EXISTS subqueries, constraints, and indexes. No recursive query is used. Execute the demo queries for live output. |
| 4 | Advanced Database Concepts | Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD | Implemented: is_moderator, review_report and merge_post functions, the report_rules trigger, the v_thread_answers view, transactional accepted-answer routes, vector search, and analytics. |
| 5 | Web Application and Database Integration | Next.js interface , Backend/API and database integration | Implemented: Express API and static HTML/CSS/JS frontend. The repository does not use Next.js. |
| 6 | Authentication and Authorization | Authentication implementation (front end and backend) and minimum two roles and role-based access control in database | Implemented: JWT authentication, bcrypt password comparison, student/teacher/admin roles, verified-teacher checks, and database-level moderator checks (is_moderator). |
| 7 | Documentation, Git/GitHub, Demonstration, Testing, and Originality | Documentation, Git/GitHub , Originality and individual contribution | Individual contributions documented. Repo contains SQL and API test scripts and a README demo guide; execute them for live evidence. |

Primary implementation references: schema.sql, seed.sql, queries.sql, queries/dashboards.sql, queries/explain_demo.sql, migrations/001_search_moderation.sql through migrations/004_teacher_mention.sql, src/routes/auth.js, src/routes/posts.js, src/routes/answers.js, src/routes/moderation.js, src/routes/search.js, src/services/searchClient.js, embed_posts.py, search_service.py, public/app.js, public/extras.js, and README.md.